1. 哪个函数增长得更快：$N\log N$ 和 $N^{1+\epsilon/\sqrt{\log N}}$，其中 $\epsilon>0$，给出具体分析。

(1) 解法一：$\lim_{N\to\infty}\frac{NlogN}{N^{1+\frac{\epsilon}{\sqrt{logN}}}}=\lim_{t\to\infty}\frac{e^t\cdot t}{(e^t)^{1+\frac{\epsilon}{\sqrt{t}}}}=\lim_{t\to\infty}\frac{t}{e^{\epsilon\sqrt{t}}}=\lim_{x\to\infty}\frac{x^2}{e^{\epsilon x}}=0$，所以$N^{1+\frac{\epsilon}{\sqrt{logN}}}$增长更快。

(2) 解法二：要证 $\mathrm{NlogN}<\mathrm{N}^{1+\frac{\epsilon}{\sqrt{\log N}}}$，即证 $\mathrm{logN}<\mathrm{N}^{\frac{\epsilon}{\sqrt{\log N}}}$，用反证法，假设 $\mathrm{N}^{\frac{\epsilon}{\sqrt{\log N}}}<\mathrm{logN}$。两边取对数有 $\frac{\epsilon}{\sqrt{\log N}}\mathrm{logN}<\mathrm{loglogN}$，即 $\epsilon\sqrt{\mathrm{logN}}<\log\log\mathrm{N}$。设 $\mathrm{t}=\log\mathrm{N}$，则 $\epsilon\sqrt{\mathrm{t}}<\log\mathrm{t}$，两边求平方有 $\epsilon^2\mathrm{t}<\log^2\mathrm{t}$，矛盾，因此假设 $\mathrm{N}^{\frac{\epsilon}{\sqrt{\log N}}}<\mathrm{logN}$ 不成立，因此函数 $\mathrm{N}^{1+\frac{\epsilon}{\sqrt{\log N}}}$ 增长得更快。

2. 考虑下列算法（我国称为秦九韶方法，西方称为 Horner 方法），计算实值函数

$$F(x)=\sum_{i=0}^{N}A_ix^i$$

(1) 按照 for 循环公式带入数值进行计算，最终结果为 401

(2) 时间复杂度分析

① 秦九韶：$F(x)=\left(\left((A_0x+A_1)x+A_2\right)x+\ldots+A_{N-1}\right)x+A_N$，总计 n 次乘法 n 次加法，时间复杂度为$O(N)$

② 直接求幂：总计$\sum_{i=1}^{N+1}i$次，时间复杂度$O(N^2)$

3. 给出有效的算法来确定在递增的整数序列 $A_1<A_2<\cdots<A_N$ 的数组中，是否存在整数

(1) 1 遍历整个序列，对应时间复杂度 O(N)

(2) 2 二分法查找，对应时间复杂度 O(logN)

4. 填空

(1) 线性表

(2) head->next==null

(3) $O(1)$,$O(n)$

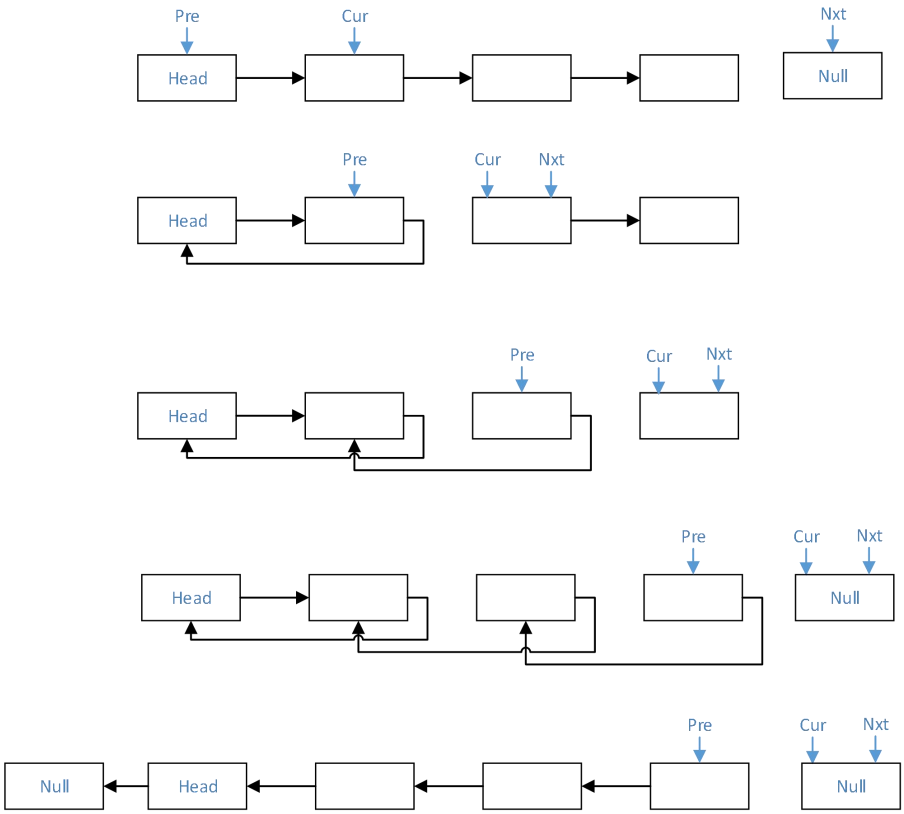
(4) 删除 x 的 next 节点

(5) 将节点 t 插入到结点 x 后面

(6) x->next=t 使得 x->next 的下一个节点地址丢失，t->next=x->next 实际上将 t 指回自己本身。

5.

(1) 考虑下述单向链表逆序的代码，针对链表长度为 4 的情况手绘草图分析指针的变化。



(2) 考虑双向链表逆序的代码设计。

```
Node* reverseDoublyLinkedList(Node* head) {
if (!head || !head->next) {
return head;
}

Node* prevNode = null;
Node* currNode = head;

while (currNode != null) {
// 交换节点的 prev 和 next 指针
Node* nextNode = currNode->next;
currNode->next = prevNode;
currNode->prev = nextNode;

// 移动到下一个节点
prevNode = currNode;
currNode = nextNode;
}

// 更新头节点
head = prevNode;

return head;
```

```
    }
```